

Formal Languages and Compilers (Linguaggi Formali e Compilatori)

prof. Luca Breveglieri (prof. S. Crespi Reghizzi, prof. A. Morzenti)

Written exam - 5 February 2013 - Part I: Theory

WARNING - STARTING FROM THE SUMMER SESSION (JUNE 2013), THE EXERCISES ON SYNTAX ANALYSIS MUST BE CARRIED OUT BY THE THEORY *ELR* / *ELL* - THE THEORY *LR* / *LL* WILL NOT BE ACCEPTED ANY LONGER - SEE THE COURSE WEBSITE FOR DOCUMENT AND INFORMATION

NAME:

SURNAME:

ID:

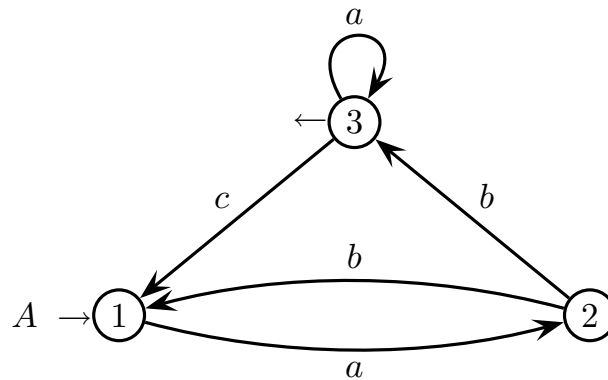
SIGNATURE:

INSTRUCTIONS - READ CAREFULLY:

- The exam consists of two parts:
 - I (80%) Theory:
 1. regular expressions and finite automata
 2. free grammars and pushdown automata
 3. syntax analysis and parsing
 4. translation and semantic analysis
 - II (20%) Practice on Flex and Bison
- To pass the exam, the candidate must succeed in both parts (I and II), in one call or more calls separately, but within one year.
- To pass part I (theory) one must answer the mandatory (not optional) questions. Notice the full grade is achieved by answering the optional questions.
- The exam is open book (texts and personal notes are admitted).
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets nor replace the existing ones.
- Time: Part I (theory): 2h.15m - Part II (practice): 60m

1 Regular Expressions and Finite Automata 20%

1. Consider the ternary alphabet $\{a, b, c\}$ and the nondeterministic automaton A below:



Answer the following questions:

- (a) By means of the Berry-Sethi method, from automaton A obtain an equivalent deterministic and minimal automaton A' .
 - (b) By means of a method of choice (e.g., node elimination or language equations), from automaton A obtain an equivalent regular expression R .
 - (c) (optional) Letter b is canceled from the strings of the language (e.g., transition $2 \xrightarrow{b} 3$ becomes $2 \xrightarrow{\varepsilon} 3$, etc); the rest is left unchanged. Quickly find a deterministic automaton (minimal or not) for this new modified language.
-

2 Free Grammars and Pushdown Automata 20%

1. One wishes one defined the grammars for some extensions of the classical free language $a^n c b^n$ ($n \geq 1$). Thus answer the following questions:

- (a) The strings of a language L contain an even number ≥ 4 of non-empty letter groups; every group only contains either letters a or letters b ; and the groups are separated by isolated letters c (notice the number of letters c is odd). The groups of a and of b alternate, starting with a group of a followed (after a c) by a group of b followed (after a c) by a group of a , and so on as far as the last group, formed by b . The groups of a and of b that are placed in the symmetrical positions with respect to the central c , which means the first and last groups, the second and penultimate ones, etc, have the same length.

Sample string $\in L$:

$$a^2 c b^3 c a^3 c b^2$$

Sample strings $\notin L$:

$$a^3 c b^2 c a^3$$

$$a^2 c b^3 c a^2 c b^2$$

If possible, write a *BNF* grammar (not extended) for language L .

- (b) Consider a language L_1 variant of L , the strings of which contain groups of a and of b in the same number as in L . The groups alternate like in L , but in such a way that the length of each group of a is strictly greater than the one of the corresponding group of b that is placed in the symmetrical position with respect to the central c ; which means that the first group is strictly longer than the last one, the penultimate is strictly longer than the second, etc.

Sample string $\in L_1$:

$$a^3 c b c a^2 c b^2$$

Sample strings $\notin L_1$:

$$a^2 c b^2 c a^3 c b^2$$

$$a^3 c b^3 c a^2 c b^2$$

If possible, write a *BNF* grammar (not extended) for language L_1 .

- (c) (optional) Consider one more variant L_2 of language L , the strings of which contain groups of a and of b in the same number as in L . The groups alternate like in L , but in such a way that all the groups of a and of b exactly have the same length.

Sample string $\in L_2$:

$$a^3 c b^3 c a^3 c b^3$$

If possible, write a *BNF* grammar (not extended) for language L_2 .

2. Here follows a part of the official specification of the language Lua, which is often used in (video)game programming. Lua's syntax is *EBNF* (axiom `stat`); the graph brackets mean that a construct may be missing or be repeated one or more times.

```

stat    ::= varlist '=' explist
varlist ::= var {',' var}
explist ::= {exp ','} exp
exp     ::= nil | false | true | Number | String | exp binop exp
           | unop exp
binop   ::= '+' | '-' | '*' | '/' | '^' | '..' | '<' | '=='
           | and | or
unop    ::= '-' | not | '#'

```

Lua supports the usual arithmetic operators: the binary '+' (addition), '-' (subtraction), '*' (multiplication) and '/' (division); '^' (exponentiation); and the unary '-' (negation).

The relational operators to be considered are: '==' and '<'.

The logical operators are: 'and', 'or' and 'not'.

The string concatenation operator is denoted by two dots '..'.

The length (of string) operator is denoted by the unary operator '#'.

Some examples of statement:

```

x = 10 or 20 and - y
y, z = nil and (10 or x), 7 + #'abc' * w^3

```

The grammar shown above is ambiguous and does not assign any precedence to the operators; rather, Lua's manual specifies them separately, and furthermore specifies which operators are right associative and which ones are left associative, as follows:

Operator precedence follows the table below from lower to higher priority:

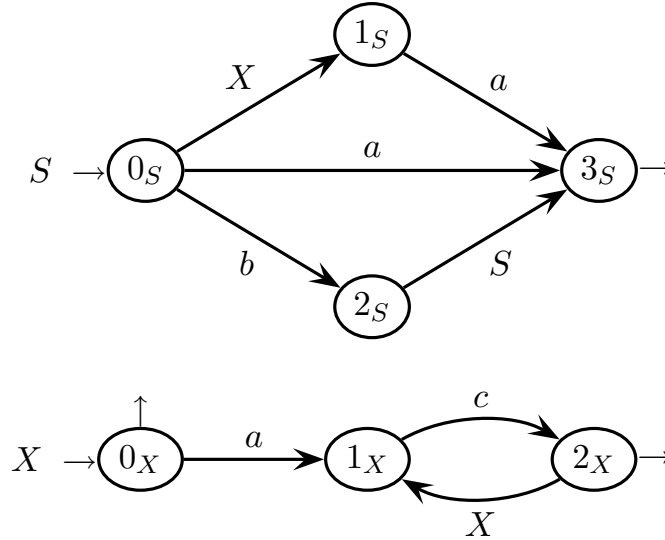
or		
and		
<	==	
..		
+	-	
*	/	
not	#	- (unary)
^		

As usual, you can use parentheses to change the precedences of an expression. The concatenation '..' and exponentiation '^' operators are right associative. All other binary operators are left associative.

By modifying the grammar shown above, write a *BNF* grammar (not extended) to introduce brackets (round), to assign to the operators the precedences listed before, and to make each operator either right or left associative as specified before.

3 Syntax Analysis and Parsing 20%

1. Consider the grammar specified below, represented as a network of recursive machines (axiom S):



Answer the following questions:

- (a) Draw the network pilot graph and say if the grammar is of type *ELR*, with listing the possible conflicts and describing them shortly.
- (b) Draw the network control flow graph *PCFG* along with the prospect sets (in the final states) and the guide sets (on the call edges and the exit darts), and say if the graph is deterministic or not, with listing the possible nondeterminism points and describing them shortly. Furthermore say if the pilot graph satisfies the Single Transition Property (*STP*) and explain why.
- (c) (optional) The letter a from 1_S to 3_S becomes a c and 0_X is not final any longer; the rest of the network is left unchanged. Is the grammar now *ELR*? Is it now *ELL*? Shortly explain your answers.

FOR WHAT FOLLOWS SEE THE WARNING ON THE TEXT COVER PAGE

Those who prefer to answer (a-b) by means of the classical *LR* and *LL* analysis methodologies, please use the not extended (*BNF*) grammar below (axiom S).

$$\left\{ \begin{array}{l} S \rightarrow X a \mid b S \mid a \\ X \rightarrow a c Y \mid \varepsilon \\ Y \rightarrow X c Y \mid \varepsilon \end{array} \right.$$

To answer (c) according to the classical *LR* and *LL* analysis methodologies, apply the specified changes to the *BNF* grammar.

4 Translation and Semantic Analysis 20%

1. The grammar G below (axiom S):

$$G: S \rightarrow (S) S \mid \varepsilon$$

generates the Dyck language with round brackets '(' and ')'. One wishes one wrote a transduction scheme τ , where the round brackets at a nesting depth n ($n \geq 1$) are transliterated into graph brackets if the relationship $n \bmod 3 = 1$ holds, into square brackets if $n \bmod 3 = 2$ holds, while if it holds $n \bmod 3 = 0$ then the round brackets are left unchanged.

Notice: the outermost round brackets of the source string are supposed to be at a nesting depth 1.

For instance the source string x :

$$x = (()) (((())))$$

is translated into the destination string $\tau(x)$:

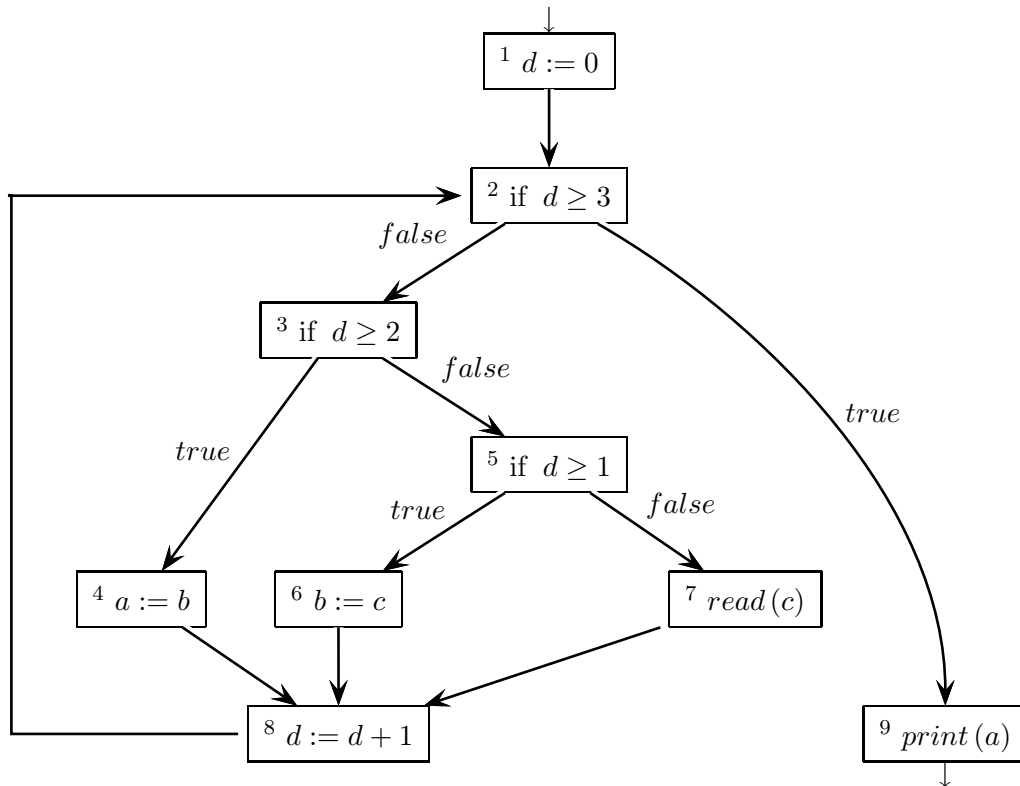
$$\tau(x) = \{ [] \} \{ [] [(\{ \})] \}$$

Answer the following questions:

- (a) By modifying the grammar G shown above, write the source grammar G_s of a transduction scheme that defines the translation τ described before.
- (b) Complete the transduction scheme τ described above and add the destination part G_d to the source grammar G_s obtained before.
- (c) (optional) Draw the destination syntax tree of the translation τ of the sample source string x shown above:

$$x = (()) (((())))$$

2. In the program below, represented by means of its Control-Flow Graph, one wishes one made some static analyses, which are described in the following.



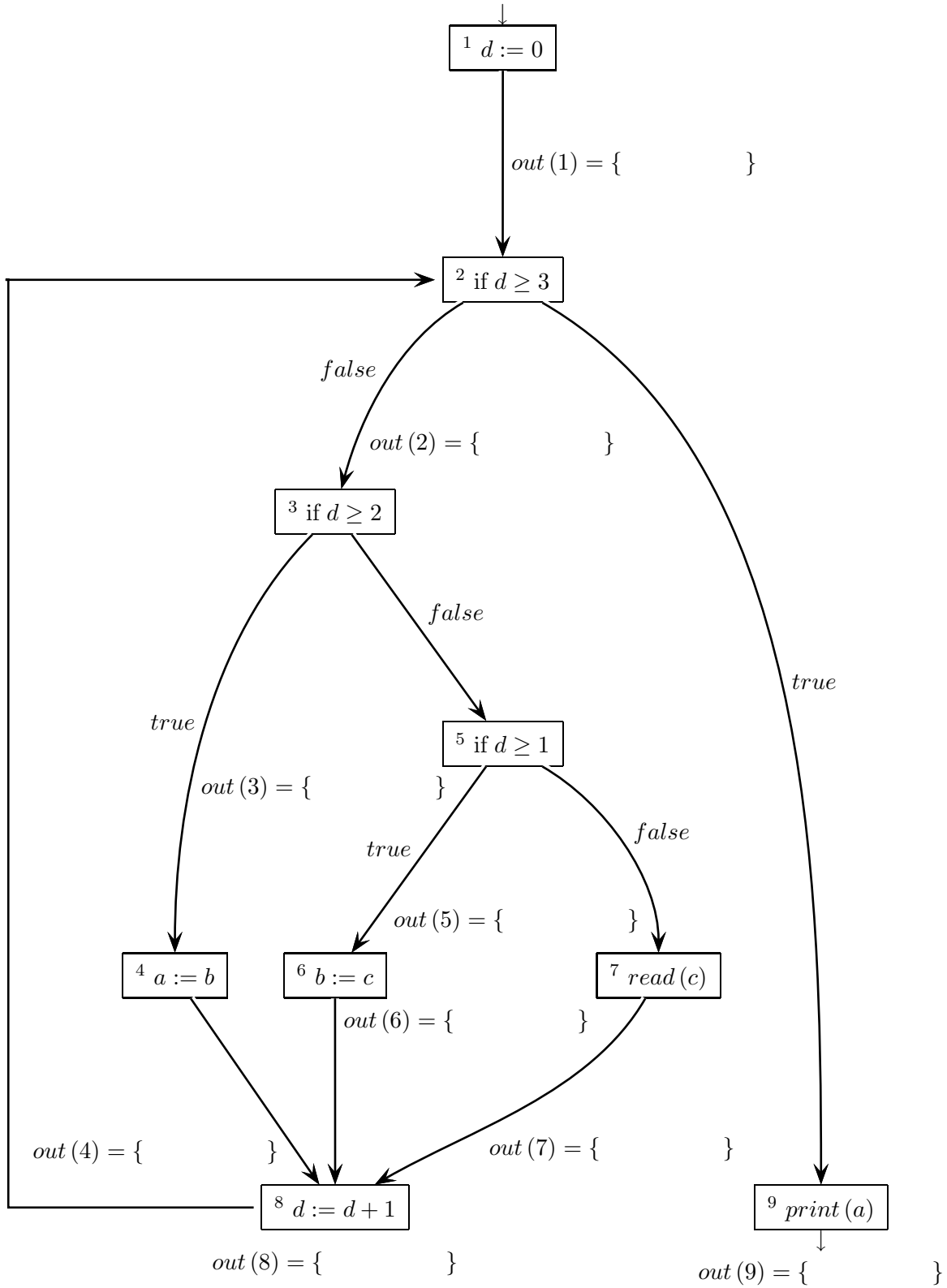
Answer the following questions:

- (a) Write, in the space prepared on the next pages, the graph flow equations to compute the reaching definitions on the output of node 8 in the graph. Furthermore compute all the sets of reaching definitions on the outputs of the graph nodes. Write such sets in the figure prepared on the next pages.
- (b) (optional) One wishes one checked if there are some *Possibly Uninitialized* (or *PU*) variables. A variable x is *PU* at a point u of the graph, if there is a path from the initial node of the graph to point u , on which:
 - i. either there is not any definition of variable x
 - ii. or the definition of variable x is of the type $x \leftarrow e$, where the expression e uses a variable that is *PU*

For instance, variable b is *Possibly Uninitialized (PU)* at point *out*(4) because on the path 1, 2, 3, 4 there is not any definition of b . Instead, variable d is not *PU* at *out*(4) because every path from 1 to 4 contains a definition of d . And variable b is *PU* at *out*(6) because of path 1, 2, 3, 5, 6, which contains definition $b \leftarrow c$ where c appears to be itself *PU*.

In the table prepared on the next pages, write the sets of *PU* variables, on the input and output of each graph node.

write in the figure the sets of reaching definitions



write here the requested flow equations (for node 8)

write in the table the sets of Potentially Undefined (PU) variables

<i>node</i>	<i>in</i>	<i>out</i>
1		
2		
3		
4		
5		
6		
7		
8		
9		

